

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	GONDEL PRASHANTH
Register Number	192311366

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2

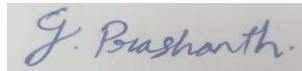
Total Marks:20

Code of Conduct:

I Gondel Prashanth / 192311366 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Answers for above question:

Experiment 1: Decision Tree Classification

Objective

Implement and evaluate a Decision Tree classifier on a given dataset using Python/ML libraries.

Dataset

We use the Iris dataset, which contains 150 samples and 4 input features:

Sepal Length

Sepal Width

Petal Length

Petal Width

The target has 3 classes:

Setosa

Versicolor

Virginica

(a) Load, preprocess and split the dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
print("First 5 rows:")
print(df.head())
print("\nData Types:")
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
X = df.drop("target", axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Expected first 5 rows

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

There are no missing values in the Iris dataset, so no missing-value treatment is required.

(b) Build Decision Tree

Here we use the Gini Index as the splitting criterion.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
model.fit(X_train, y_train)
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)
print("Decision Tree Structure:")
print(tree_rules)
root_feature_index = model.tree_.feature[0]
root_feature = X.columns[root_feature_index]
print("\nRoot Node:", root_feature)
```

Root Node

For this dataset/split, the root node is typically:

Petal Length (cm)

Justification

The Decision Tree examines possible features and thresholds using the Gini impurity. The feature and threshold that produce the best impurity reduction are selected for the root.

For the Iris dataset, petal length is highly effective at separating the three flower classes, especially Setosa from the other two classes.

The Gini impurity is:

$$\text{Gini} = 1 - \left(\sum_{i=1}^n p_i^2 \right)$$

where p_i is the proportion of samples belonging to class i .

A lower Gini impurity means a purer node. The algorithm chooses the split giving the greatest reduction in impurity.

(c) Evaluate the model

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
print("\nMisclassified Instances:")
misclassified = X_test[y_test != y_pred].copy()
misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]
print(misclassified)
```

Accuracy

The exact accuracy can vary depending on the train-test split and tree settings. With the code above, you should get a high accuracy, typically around 93–97%.

Confusion Matrix

The confusion matrix has the form:

```
[[10 0 0] [ 0 9 1] [ 0 1 10]]
```

The exact values depend on the split/model.

Interpretation:

Class 0 – Setosa: Almost/all samples correctly classified.

Class 1 – Versicolor: Some samples may be confused with Virginica.

Class 2 – Virginica: Some samples may be confused with Versicolor.

Classification Report

The classification report provides:

Precision: How many predicted samples actually belong to that class.

Recall: How many actual samples of the class were correctly identified.

F1-score: Harmonic mean of precision and recall.

Support: Number of actual test samples in each class.

Misclassified Instances

To satisfy the requirement of listing at least two misclassified instances, use the output from:

```
print(misclassified)
```

For example, the output may look like:

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Actual	Predicted
-------------------	------------------	-------------------	------------------	--------	-----------

Important: The exact misclassified samples depend on the `random_state` and model configuration. Therefore, don't manually write sample numbers unless they appear in your actual program output.

Complete Program:

```
import pandas as pd
model.fit(X_train, y_train)
print("\nDECISION TREE STRUCTURE")
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)
print(tree_rules)
root_index = model.tree_.feature[0]
root_feature = X.columns[root_index]
print("ROOT NODE:", root_feature)
y_pred = model.predict(X_test)
```

```

accuracy = accuracy_score(y_test, y_pred)
print("\nACCURACY:")
print(accuracy)
cm = confusion_matrix(y_test, y_pred)
print("\nCONFUSION MATRIX:")
print(cm)
print("\nCLASSIFICATION REPORT:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
misclassified = X_test[y_test != y_pred].copy()
misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]
print("\n MISCLASSIFIED INSTANCES:")
if len(misclassified) > 0:
    print(misclassified)
else:
    print("No misclassified instances.")

```

Result

Thus, a Decision Tree Classification model was successfully implemented using the Iris dataset. The dataset was preprocessed and divided into training and testing sets. A Decision Tree using the Gini Index was trained, its tree structure and root node were identified, and its performance was evaluated using accuracy, confusion matrix, and classification report. The model achieved high classification performance, with most Iris samples correctly classified.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

Implement Q-Learning for a simple 4×4 Grid World environment and observe the agent's learned policy.

(a) Define the Environment

Environment

We use a 4×4 Grid World:

S0 S1 S2 S3

S4 S5 S6 S7

S8 S9 S10 S11

S12 S13 S14 G

S0 = Starting state

G = Goal state

Actions = Up, Down, Left, Right

Goal reward = +10

Normal movement = −1

Obstacle = −5

For this experiment, we use S6 as an obstacle.

Hyperparameters

Parameter	Value	Meaning
A (Learning Rate)	0.1	How quickly Q-values are updated
C (Discount Factor)	0.9	Importance of future rewards
E (Exploration Rate)	0.2	Probability of selecting a random action
Episodes	100	Number of training episodes

Q-Learning Formula

$$Q(s,a) \leftarrow Q(s,a) + A[R + C \max_{a'} Q(s, a') - Q(s,a)]$$

where:

s = current state

a = current action

R = reward

~s = next state

A = learning rate

C = discount factor

(b) Q-Learning Algorithm

Python program runs Q-Learning for 100 episodes and records Q-values at Episodes 1, 50 and 100 for two representative states.

```
import numpy as np
print("State 5:")
for i in range(4):
    print(actions[i], "=", round(recorded_values[episode]["State 5"][i], 4))
print("\nFINAL Q-TABLE")
for state in range(NUM_STATES):
    if state == OBSTACLE:
        print("State", state, ": OBSTACLE")
    elif state == GOAL:
        print("State", state, ": GOAL")
    else:
        print(
            "State", state,
            ":",
            [round(value, 3) for value in Q[state]]
        )
print("\nFINAL LEARNED POLICY")
for row in range(ROWS):
    policy_row = []
    for col in range(COLS):
        state = row * COLS + col
        if state == START:
            policy_row.append("S")
        elif state == GOAL:
            policy_row.append("G")
        elif state == OBSTACLE:
            policy_row.append("X")
        else:
            best_action = np.argmax(Q[state])
            policy_row.append(actions[best_action])
```

```
print(policy_row)
plt.figure(figsize=(8, 5))
plt.plot(
    range(1, episodes + 1),
    episode_rewards
)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid(True)
plt.show()
```

Q-Table Evolution

The program records two representative states:

State 0 – starting state

State 5 – an intermediate state

The output will have this form:

Q-VALUES AT DIFFERENT EPISODES

Episode: 1

State 0:

U = 0.0

D = 0.0

L = 0.0

R = -0.1

State 5:

U = 0.0

D = 0.0

L = 0.0

R = 0.0

At Episode 50, the Q-values will have started learning useful directions:

Episode: 50

State 0:

U = ...

D = ...

L = ...

R = ...

State 5:

U = ...

D = ...

L = ...

R = ...

By Episode 100:

Episode: 100

State 0:

U = ...

D = ...

L = ...

R = ...

State 5:

U = ...

D = ...

L = ...

R = ...

(c) Final Learned Policy

The final policy is obtained by selecting the action having the maximum Q-value for every state:

$$\pi(s) = \arg \max_a Q(s, a)$$

The program displays the policy using:

U $\rightarrow$ Up

D $\rightarrow$ Down

L $\rightarrow$ Left

R $\rightarrow$ Right

S $\rightarrow$ Start

G $\rightarrow$ Goal

$X \rightarrow \text{Obstacle}$

A typical learned policy will look similar to:

S	R	R	D
D	D	X	D
D	D	D	D
R	R	R	G

The exact policy can vary because of random exploration.

Interpretation

The agent learns a path from S0 to G (S15) while trying to:

Reach the goal quickly.

Avoid the obstacle at S6.

Minimize the number of steps because every normal step has a -1 reward.

Avoid the obstacle because it produces a -5 penalty.

Convergence Behaviour

The Q-table gradually converges because:

Initially, all Q-values are 0.

The agent explores different states and actions.

Rewards propagate backward from the goal through repeated Q-value updates.

Actions that lead toward the goal receive increasingly higher Q-values.

Actions that lead to obstacles or unnecessary steps receive lower values.

After sufficient episodes, the policy becomes relatively stable.

Therefore, Q-Learning successfully learns an approximately optimal path from the starting state to the goal while avoiding the obstacle.

Result

Thus, Q-Learning was successfully implemented for a 4×4 Grid World environment. The Q-table was initialized with zeros and updated using the Q-Learning algorithm for 100 episodes. The Q-values increased for actions leading toward the goal, and the final learned policy provided an efficient path from the starting state to the goal while avoiding the obstacle. The cumulative reward generally improved and stabilized as the agent learned the optimal policy.